

Building a GIS Digital Twin for a US City — From Scratch

Reference project (what you uploaded): a Next.js 15 + React 19 app using Mapbox GL JS 3 for the base 3D map, a custom Three.js layer for photoreal building meshes, CesiumJS as an alternate globe engine, and Mapbox Directions for routing — all built around Manhattan Community District 1 (MN01), with data as static GeoJSON + GLB files under `public/data/`.

You want to rebuild this pattern for a different US city, using free data, with your own pipeline. Below is a full path from zero to a working, deployable twin — plus how to go further with the PostGIS/GDAL/GeoServer stack you already use at DHA.

Phase 0 — Pick the city

Not every US city has equally good open data. Filter by three things: (1) a public GIS/open-data portal with building footprints, (2) LiDAR coverage in USGS 3DEP for terrain/heights, (3) decent OpenStreetMap building/POI density.

Strong candidates:

City	Why it's good
San Francisco	DataSF portal, very clean building footprints + zoning, dense OSM coverage, compact area good for a first build
Seattle	Seattle GeoData portal, excellent LiDAR, city publishes 3D building data
Chicago	Very mature Open Data Portal (Socrata), building footprints with height attributes already included
Boston	Boston's Analyze Boston portal has 3D building models (LOD1/2) ready-made
Denver	Denver Open Data Catalog, good LiDAR, less dense so easier to test with

If this is your first time doing this end to end, I'd pick a **small, well-documented district within one of these** (like you did with MN01) rather than a whole city — same reasoning the original project used.

Phase 1 — Data sources (all free)

This is the part that differs most from the NYC build, so it's worth mapping out before writing any code.

Building footprints + heights

City's own open data portal (best — often includes height/floor count)

[Microsoft US Building Footprints](#) — nationwide, footprints only, no height

OpenStreetMap buildings (via Overpass API or Geofabrik extracts) — variable height tags

Photorealistic 3D (this replaces the hand-built GLB tiles)

Google Photorealistic 3D Tiles via Cesium ion — this is the practical upgrade over what the reference project did. Cesium ion has a free tier that gives you access to Google's photorealistic 3D tiles for basically any US city, streamed directly — no manual mesh generation, no per-building GLB export. This is almost certainly a better foundation than recreating MN01's manual photoreal pipeline.

If you want to keep a hand-built-mesh option too: USGS 3DEP LiDAR → building extrusion via `PDAL`/`GDAL` → Blender or `three.js` tooling to bake GLBs (heavier, optional, "advanced" path).

Terrain / elevation

USGS 3DEP 1-meter DEM (nationwide, free, via The National Map)

Streets, parks, water, transit

City open data portal, or OpenStreetMap extract (fastest single source for all of these at once)

Routing

Mapbox Directions API (same as reference project — free tier is generous) or OSRM self-hosted if you want it fully open-source

Weather

OpenWeather free tier (same as reference project)

Phase 2 — Data processing pipeline

This is where your existing GDAL/PostGIS/GeoServer skills carry over directly — and where I'd suggest you actually go *beyond* the reference project's design.

The reference project just ships static GeoJSON files in `public/data/`. That's fine for a fixed small district, but it doesn't scale and isn't really a "digital twin" in the operational sense. Since you already run PostGIS + GeoServer at DHA, consider:

Ingest raw city/OSM data into PostGIS (`ogr2ogr -f PostgreSQL`)

Normalize CRS to EPSG:4326 for web delivery (reproject from whatever state plane / local CRS the source uses — same pattern as MN01's EPSG:2263 → 4326 conversion)

Compute derived attributes in PostGIS: building height from floor count or LiDAR max Z minus ground Z, footprint area, etc.

Serve dynamically via GeoServer (WFS/WMS, or WFS as GeoJSON) instead of static files — lets you update the dataset without redeploying the frontend

Or, for a lighter first version, export processed layers to static GeoJSON (mirrors the reference project exactly) and add the PostGIS/GeoServer layer later once the frontend works

I'd actually build it in that order: static GeoJSON first (get the map working end-to-end fast), then swap the data layer for PostGIS/GeoServer once the app is stable. That matches how the reference project itself is structured (`public/data/*.geojson`) and de-risks the frontend work.

Phase 3 — App skeleton (mirrors the reference architecture)

Next.js 15 + React 19 (`create-next-app`)

Mapbox GL JS 3 for the default 3D map + building extrusions + routing

CesiumJS as an alternate engine, using Cesium ion for Google Photorealistic 3D Tiles instead of custom GLBs

Keep the reference project's env-var pattern: `NEXT_PUBLIC_MAPBOX_ACCESS_TOKEN`, `NEXT_PUBLIC_CESIUM_ION_TOKEN`

(new), `OPENWEATHER_API_KEY` (optional)

Same data contract idea: `buildings.geojson`, `meta.json`, `parks.geojson`, `water.geojson`, `roads.geojson`, etc. under `public/data/`

Phase 4 — Core features to build in order

Base map load + camera framing for your chosen district

Building footprints as 3D extrusions (fast win, no external 3D tiles needed yet)

Swap in Google Photorealistic 3D Tiles via Cesium ion for the "wow" layer

Overlay layers: parks, water, roads, transit

Routing (walk/drive/bike) via Mapbox Directions

Weather widget (optional, same as reference)

HUD/dock UI

Phase 5 — Going further (beyond the reference project)

Since you specifically want to go further than MN01's static-file approach:

Live data feeds: 311 requests, permits, or transit positions from the city's real-time open data APIs, streamed onto the map

Analysis layer: sun/shadow studies, viewshed analysis, flood/elevation overlays — all doable with the PostGIS + DEM data you'll already have ingested

Multi-city support: parametrize the data pipeline so a new city just needs a new PostGIS schema + config, not new code

Auth/access control if this becomes something you deploy for others (DHA-relevant pattern given your Django background — could even front the API with Django REST instead of/alongside GeoServer)

Suggested first milestone

Don't try to build all of this at once. A realistic first milestone: **pick one small district in one city, get building footprints + terrain rendering as a 3D Mapbox scene with routing working, deployed locally**. That alone recreates the core of the reference project. Photorealistic tiles, PostGIS backend, and live data layers are all additive after that works.

If you want, I can help you start with Phase 0/1 concretely — e.g. pick a specific city and pull the actual building footprint + LiDAR data for it right now, or scaffold the Next.js app structure.